

## Unit-3

### Context free Grammars

#### Formal Grammar:

- A grammar is a notation for defining a language through finite number of rules.

#### Types of Grammars:

- Mainly the grammars are classified into two types:
  - i) Linear
  - ii) Non-Linear

#### Linear Grammar:

- A grammar, with at most one variable (non-terminal) at right side of production, is a linear grammar.

Ex:

$$S \rightarrow aSb$$
$$A \rightarrow aAb$$

#### Non-Linear Grammar:

- A grammar, with more than one variable at right side of production is non-linear grammar.

Ex:

$$S \rightarrow AS$$
$$S \rightarrow BAS$$

## Right and Left - Linear grammars :

- Linear grammars are further classified into :

- i) Right Linear grammar (RLG)
- ii) Left Linear grammar (LLG)

### Right Linear Grammar (RLG) :

- When variable is present in the "right most" side in the right side of the production, it is called as RLG.

Ex:

$$A \rightarrow xB$$

$$S \rightarrow aBS$$

### Left Linear Grammar (LLG) :

- When variable is present in "left-most" side in the right side of production, it is called as LLG.

Ex:

$$A \rightarrow Bx$$

$$S \rightarrow ABS$$

## Regular Grammar:

- A grammar is said to be regular, if it is either RLG or LLG grammar.

Ex:

$$S \rightarrow AB$$

$$S \rightarrow SA$$

## Non-Regular Grammar:

- A grammar is said to be non-regular, if it is neither RLG nor LLG

Ex:

$$S \rightarrow aAb$$

$$S \rightarrow aSb$$

## Recursive Grammar:

- A grammar is said to be recursive grammar if it either recursive production or indirectly recursive production.

Ex:

$$S \rightarrow aS$$

(Recursive grammar)

$$S \rightarrow b|aA$$

$$A \rightarrow c|bS$$

(Indirect Recursive grammar)

## Context Free Grammar : (CFG)

- A context free grammar is in the form

$\&$  :

$$G = \{V, T, P, S\}$$

where  $V$  - Variables ( $\&$ ) Non-Terminals  
(Represented by upper case letters)  
Ex:  $S, A, B$

$T$  - Terminals ( $\&$ ) Symbol  
(Represented by lower case letters)  
Ex:  $a, b$

$P$  - production

(which is in form of  $\alpha \rightarrow \beta$ )  
Ex:  $S \rightarrow a$   $\quad \quad \quad \begin{matrix} \uparrow & \uparrow \\ \text{(L.H.S)} & \text{(R.H.S)} \end{matrix}$   
 $S \rightarrow aB$   
 $A \rightarrow aA$

$S$  - starting Symbol

(which is subset of Variables)

Ex:  $\begin{cases} A \rightarrow aB \\ S \rightarrow b \end{cases}$

Here both production form single grammar. So here starting symbol of above grammar is 'A'

Ex: Construct grammar that accepts a language where 'a' as has all possible combinations.

Sol: Regular Expression =  $a^*$

$$\text{Language (L)} = \{ \epsilon, a, aa, aaa, \dots \}$$

Construct Grammar:

$$\underline{\alpha} \rightarrow \underline{\beta}$$

$$\begin{cases} S \rightarrow aS \\ S \rightarrow \epsilon \end{cases}$$

Check above grammar is correct or not

$$\begin{aligned} S &\rightarrow aS & S &\rightarrow aS | \epsilon \\ S &\rightarrow \epsilon & (8) \end{aligned}$$

Now replace variables with terminals to obtain any string that belongs to above language 'L'

$$S \rightarrow aS | \epsilon$$

$$S \Rightarrow a \underline{a} S \quad [\because S \rightarrow aS]$$

$$S \Rightarrow a a \underline{a} S \quad [\because S \rightarrow aS]$$

$$S \Rightarrow a a a \epsilon \quad [\because S \rightarrow \epsilon]$$

Now obtained string "aaa" present in above language.

So obtained grammar is correct

$$\begin{aligned} \text{CFG} &= \{ V, T, P, S \} \\ &= \{ \{ S \}, \{ a \}, S \rightarrow aS | \epsilon, S \} \end{aligned}$$



Ex: Construct grammar for 'a' which have all combination except 'ε'

Sol:

$$P.E = a^+$$

$$L = \{ a, aa, aaa \dots \}$$

Construct Grammar:

$$\underline{S} \rightarrow \underline{\beta}$$

$$\begin{cases} S \rightarrow as \\ S \rightarrow a \end{cases} \quad (\text{or}) \quad S \rightarrow as|a$$

Check whether string 'aaa' supports above grammar or not

$$S \rightarrow as|a$$

$$S \Rightarrow aas \quad [\because S \rightarrow as]$$

$$S \Rightarrow aaa \quad [\because S \rightarrow a]$$

Hence string is obtained using above grammar

$$CFG = \{ V, T, P, S \}$$

$$= \{ \{s\}, \{a\}, S \rightarrow as|a, S \}$$

Ex: Construct grammar for  $L = a^n b^n \mid n \geq 1$

Sol:  $L = \{ ab, aabb, aaabbb, \dots \}$

CFG Construction:

$\alpha \rightarrow \beta$

$A \rightarrow aAb$

(2)  $A \rightarrow aAb \mid ab$

$A \rightarrow ab$

Check aaabbb:

$A \rightarrow aAb$

$A \Rightarrow a \underline{aAb} b \quad [\because A \rightarrow aAb]$

$A \Rightarrow a a \underline{ab} bb \quad [\because A \rightarrow ab]$

$CFG = \{ \{A\}, \{a\}, A \rightarrow aAb \mid ab, \{A\} \}$

Ex: Construct grammar for  $(a+b)^*$

Sol:  $L = \{ \epsilon, a, b, ab, aa, bb, abba, \dots \}$

$\alpha \rightarrow \beta$

$S \rightarrow as \mid bs \mid \epsilon$  (2)

$S \rightarrow as$

$S \rightarrow bs$

$S \rightarrow \epsilon$

check ab:

$S \rightarrow aS$

$S \Rightarrow a \underline{bS} \quad [\because S \rightarrow bS]$

$S \Rightarrow ab \underline{\epsilon} \quad [\because S \rightarrow \epsilon]$

$S \Rightarrow ab$

Ex: Construct grammar for  $a^i b^j c^k \mid i=j \text{ (or) } j=k$

Sol: Let us consider,

If  $i=j=n$ , then

If  $j=k=m$ , then

$$\text{Language } (L_1) = \frac{a^n b^n c^k}{A_1 \quad B_1}$$

$$\text{Language } (L_2) = \frac{a^i b^m c^m}{A_2 \quad B_2}$$

where  $L = L_1 \cup L_2$

Grammar for  $L_1$ :

$$S_1 \rightarrow A_1 B_1$$

$$A_1 \rightarrow a A_1 b \mid ab$$

$$B_1 \rightarrow B_1 c \mid c$$

Grammar for  $L_2$ :

$$S_2 \rightarrow A_2 B_2$$

$$A_2 \rightarrow a A_2 \mid a$$

$$B_2 \rightarrow b B_2 c \mid bc$$

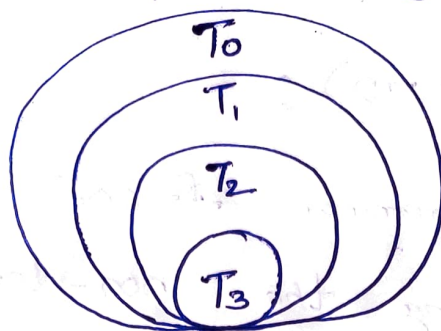
combine

$$\boxed{S \rightarrow S_1 \mid S_2}$$



# Hierarchy of Grammar / Classification of Grammar

- This hierarchy of grammar (or) classification of grammar is proposed by "Noam Chomsky"



where  $T_3 \subset T_2 \subset T_1 \subset T_0$

| Non Restrictive Grammar ( $T_0$ )                                                                    | Context Sensitive Grammar ( $T_1$ )                                                                                          | Context free Grammar ( $T_2$ )                                                                             | Regular Grammar ( $T_3$ )                                                                                       |
|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Supports the Recursive enumerating Language                                                          | Supports the context sensitive Language                                                                                      | Supports the context free Language                                                                         | Supports the Regular language                                                                                   |
| Supported by Turing Machine (TM)                                                                     | Supported by Linear bounded Automata                                                                                         | Supported by push down Automata (PDA)                                                                      | Supported by Finite Automata (FA)                                                                               |
| $\alpha \rightarrow \beta$<br>$\alpha, \beta \in (V \cup T)^*$<br><u>ex:</u> $SABa \rightarrow bcDE$ | $\alpha \rightarrow \beta$<br>$\alpha, \beta \in (V \cup T)^+$<br>$ \alpha  \leq  \beta $<br><u>ex:</u> $aB \rightarrow SBA$ | $\alpha \rightarrow \beta$<br>$\alpha \in V$<br>$\beta \in (V \cup T)^*$<br><u>ex:</u> $B \rightarrow aBS$ | $\alpha \rightarrow \beta$<br>$\alpha \in V$<br><u>ex:</u> $S \rightarrow aA$ (RLG)<br>$S \rightarrow Aa$ (LLG) |

## Left most & Right most derivation :

- In grammar  $\rightarrow$  is production  
where  $\Rightarrow$  is derivation

### Left Most Derivation (LMD) :

- Expanding the grammar (or) deriving the grammar by replacing the non-terminals with terminals only on "left side" achieves LMD.

Derive string : 00101

Ex :

$$S \rightarrow AIB$$

$$A \rightarrow OA|E$$

$$B \rightarrow OB|IB|E$$

$$S \Rightarrow \underline{A}IB$$

$$S \Rightarrow O\underline{A}IB \quad [ \because A \rightarrow OA ]$$

$$S \Rightarrow OOA\underline{I}B \quad [ \because A \rightarrow OA ]$$

$$S \Rightarrow OOE\underline{I}B \quad [ \because A \rightarrow E ]$$

$$S \Rightarrow OOI\underline{B}$$

$$S \Rightarrow OOI\underline{O}B \quad [ \because B \rightarrow OB ]$$

$$S \Rightarrow OOI\underline{O}IB \quad [ \because B \rightarrow IB ]$$

$$S \Rightarrow OOI\underline{O}IE \quad [ \because B \rightarrow E ]$$

$$S \Rightarrow 00101$$

### Right Most Derivation (RMD) :

- Expanding the grammar (or) deriving the grammar by replacing the non-terminals with terminals only on "Right side" achieves RMD.

Ex:

$$S \rightarrow AIB$$

$$A \rightarrow 0A| \epsilon$$

$$B \rightarrow 0B|1B| \epsilon$$

Derive String: 00101

$$S \Rightarrow AIB$$

$$S \Rightarrow AIOB$$

$$S \Rightarrow AIO1B$$

$$S \Rightarrow AIO1 \epsilon$$

$$S \Rightarrow AIO1$$

$$S \Rightarrow 0AIO1$$

$$S \Rightarrow 00AIO1$$

$$S \Rightarrow 00AIO1$$

$$S \Rightarrow 00101$$

$$[\because B \rightarrow 0B]$$

$$[\because B \rightarrow 1B]$$

$$[\because B \rightarrow \epsilon]$$

$$[\because A \rightarrow 0A]$$

$$[\because A \rightarrow 0A]$$

$$[\because A \rightarrow \epsilon]$$

Parse Trees (or) Derivation trees:

- parse trees are the trees obtained by representing the derivation in tree format.

Components of parse tree:

Root node  $\rightarrow$  Variable (Starting symbol)

Leaf node  $\rightarrow$  Terminal

Intermediate node  $\rightarrow$  Variable and Terminal.

- parse trees also represented in two different ways:

i) Left most sparse trees

ii) Right most sparse trees.



Ex: Construct parse trees for given grammar  
Using RMD

Sol:

$$S \rightarrow AIB$$

$$A \rightarrow 0A|E$$

$$B \rightarrow 0B|B|E$$

Derivating string: 00101

$$S \Rightarrow AIB$$

$$S \Rightarrow AIOB$$

$$S \Rightarrow AIOIB$$

$$S \Rightarrow AIOIE$$

$$S \Rightarrow AIOI$$

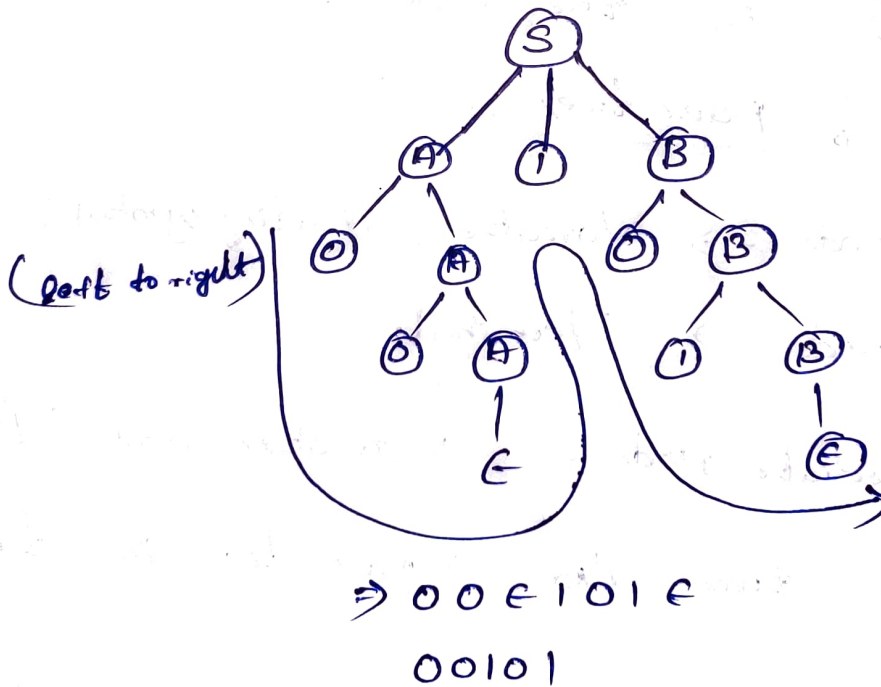
$$S \Rightarrow 0AIOI$$

$$S \Rightarrow 00AIOI$$

$$S \Rightarrow 00EIOI$$

$$S \Rightarrow 00101$$

Parse tree for above derivation:



- parse tree was constructed and string is obtained by considering "leaf nodes" from "left to right" of parse tree.

## Ambiguous Grammar:

- A grammar is said to be ambiguous, if it is deriving same string by applying or by expanding variable with different terminals.

- We may derive by using LMD or RMD, if each one is implemented in two different ways and getting same string as output called as "ambiguous grammar".

Ex.

$$S \rightarrow ABA$$

$$A \rightarrow aA | \epsilon$$

$$B \rightarrow bB | \epsilon$$

Deriving string "aa":

LMD (1-type)

$$S \Rightarrow \underline{A}BA$$

$$S \Rightarrow a\underline{A}BA$$

$$S \Rightarrow aa\underline{A}BA$$

$$S \Rightarrow aa\epsilon BA$$

$$S \Rightarrow aaBA$$

$$S \Rightarrow aa\epsilon A$$

$$S \Rightarrow aaA$$

$$S \Rightarrow aa\epsilon$$

$$S \Rightarrow aa$$

LMD (2-type)

$$S \Rightarrow \underline{A}BA$$

$$S \Rightarrow \epsilon BA$$

$$S \Rightarrow \underline{B}A$$

$$S \Rightarrow \epsilon A$$

$$S \Rightarrow \underline{A}$$

$$S \Rightarrow a\underline{A}$$

$$S \Rightarrow aaA$$

$$S \Rightarrow aa\epsilon$$

$$S \Rightarrow aa$$

- As we are getting same string by implementing different ways using LMD. So given "grammar is ambiguous".



Ex:  $E \rightarrow E + E$   
 $E \rightarrow E * E$   
 $E \rightarrow id$

Derivation string:  $id + id * id$

RMD (1-type)

$E \Rightarrow E + E$   
 $E \Rightarrow E + E * E$   
 $E \Rightarrow E + E * id$   
 $E \Rightarrow E + id * id$   
 $E \Rightarrow id + id * id$

RMD (2-type)

$E \Rightarrow E * E$   
 $E \Rightarrow E * id$   
 $E \Rightarrow E + E * id$   
 $E \Rightarrow E + id * id$   
 $E \Rightarrow id + id * id$

- Above grammar is also ambiguous, as we obtained same string as output by approaching different derivations using RMD.

Simplification of CFG:

- Simplification of CFG can be obtained by implementing below three steps:

Step 1: Remove useless symbols

Step 2: Eliminate  $\epsilon$ -production (or) Null productions

Step 3: Delete Unit productions.

## Remove Useless Symbol :

- if symbol is not present in right side of production
- (2)
- If symbol (2) variable not used for derivation, remove that symbol (2) variable.

Ex:

$$\begin{aligned} T &\rightarrow a a B \mid a b A \mid a a T \\ \times \quad A &\rightarrow a A \\ B &\rightarrow a b \mid b \\ \times \quad C &\rightarrow a \end{aligned}$$

- As 'A' not producing (2) deriving any terminal, it is useless symbol (2) variable.

- where 'C' does not used any right side of production, so it also useless symbol (2) variable.

$$\begin{cases} T \rightarrow a a B \mid a a T \\ B \rightarrow a b \mid b \end{cases}$$

(finally obtained grammar)

Ex:

$$\begin{aligned} T &\rightarrow \cancel{a a B} \mid a a \\ S &\rightarrow \overset{\times}{A} B \mid a \\ \times \quad A &\rightarrow b \end{aligned}$$

$$\{ S \rightarrow a \quad (\text{finally obtained grammar})$$

## Eliminate $\epsilon$ -production (2) Null Production :

- We eliminate null production by just replacing the  $\epsilon$ -value in given grammar.

Ex:

$$S \rightarrow amb$$

$$M \rightarrow amb$$

$$M \rightarrow \epsilon$$

Sol:

$$S \rightarrow amb|ab$$

$$M \rightarrow amb|ab$$

- Before replacing  $\epsilon$ -value we just write the grammar as it is and next replace with ' $\epsilon$ '

Ex:

$$S \rightarrow xyx$$

$$x \rightarrow ox|\epsilon$$

$$y \rightarrow oy|\epsilon$$

Sol:

eliminate  $y \rightarrow \epsilon$ ?

$$S \rightarrow xyx|xx$$

$$x \rightarrow ox|\epsilon$$

$$y \rightarrow oy|o$$

eliminate  $x \rightarrow \epsilon$ ?

$$S \rightarrow xyx|xx|y$$

$$x \rightarrow ox|o$$

$$y \rightarrow oy|o$$

## Delete Unit production

- A production is said to be unit production, when there is only one ~~non-terminal~~ (variable) on R.H.S of production.
- Unit production can be eliminated by substituting the unit production in grammar.

Ex:

$$S \rightarrow OA/IB/C$$

$$A \rightarrow OS/OO$$

$$B \rightarrow I/A$$

$$C \rightarrow OI$$

Sol: First find unit productions in above grammar

①  $S \rightarrow C$

②  $B \rightarrow A$

eliminate  $S \rightarrow C$ :

$$S \rightarrow OA/IB/OI$$

$$A \rightarrow OS/OO$$

$$B \rightarrow I/A$$

$$C \rightarrow OI$$

eliminate  $B \rightarrow A$ :

$$S \rightarrow OA/IB/OI$$

$$A \rightarrow OS/OO$$

$$B \rightarrow I/OO/OO$$

$$C \rightarrow OI$$

(Final grammar)

Ex:

$$S \rightarrow AC$$

$$A \rightarrow a$$

$$B \rightarrow D$$

$$C \rightarrow B/d$$

$$D \rightarrow E$$

$$E \rightarrow b$$

eliminate  $B \rightarrow D$ :

$$S \rightarrow AC$$

$$A \rightarrow a$$

$$B \rightarrow b$$

$$C \rightarrow B/d$$

$$D \rightarrow E$$

$$E \rightarrow b$$

eliminate  $C \rightarrow B$ :

$$S \rightarrow AC$$

$$A \rightarrow a$$

$$B \rightarrow b$$

$$C \rightarrow b/d$$

$$D \rightarrow E$$

$$E \rightarrow b$$

eliminate  $D \rightarrow E$ :

$$S \rightarrow AC$$

$$A \rightarrow a$$

$$B \rightarrow b$$

$$C \rightarrow b/d$$

$$D \rightarrow b$$

$$E \rightarrow b$$

(Final grammar)



## Chomsky Normal Form (CNF):

- A CFG can said to be CNF when follows below rules:

### Rules:

1. Starting symbol may have  $\epsilon$ -production

Ex:  $S \rightarrow \epsilon$

2. Production may contain only "one terminal" on right side of production

Ex:  $A \rightarrow b$

3. Production may have only "two non-terminals" on right side of production

Ex:  $S \rightarrow SA$

### CFG to CNF:

- To convert CFG to CNF we should follow the steps below.

### Steps:

1. In starting production, if starting is present on right side of production, then remove initial state by new production.

Ex:  $S \rightarrow aSb$

Sol:  $A \rightarrow S$   
 $S \rightarrow aSb$

2. Implement Simplification of CFG (eliminate useless symbol,  $\epsilon$ -production, unit production)

3. Replace terminals with non-terminals

ex:  $S \rightarrow aB$

sol:  $S \rightarrow AB$

$A \rightarrow a$

4. Reduce the non-terminals by replacing the non-terminal (multiple) with new non-terminal.

ex:  $S \rightarrow ACX$

sol:  $S \rightarrow BX$

$B \rightarrow AC$

Example: Convert below given CFG to CNF

$$S \rightarrow ASB \mid aB$$

$$A \rightarrow B \mid S$$

$$B \rightarrow b \mid \epsilon$$

Sol:

Step 1:

$$S \rightarrow \underline{A} \underline{S} B \mid aB$$

$$A \rightarrow B \mid S$$

$$B \rightarrow b \mid \epsilon$$

$$k \rightarrow S$$

$$\Rightarrow S \rightarrow ASB \mid aB$$

$$A \rightarrow B \mid S$$

$$B \rightarrow b \mid \epsilon$$

Step 2:

Here we didn't find useless symbols & variables to remove

Eliminate

$$\underline{B} \rightarrow \epsilon :$$

$$k \rightarrow S$$

$$S \rightarrow ASB \mid aB \mid AS \mid a$$

$$A \rightarrow B \mid S \mid \epsilon$$

$$B \rightarrow b$$

Eliminate

$$\underline{A} \rightarrow \epsilon :$$

$$k \rightarrow S$$

$$S \rightarrow ASB \mid aB \mid AS \mid a \mid SB \mid S$$

$$A \rightarrow B \mid S \mid \epsilon$$

$$B \rightarrow b$$

Eliminate

Unit production :

$$k \rightarrow S$$

$$S \rightarrow S$$

$$A \rightarrow B$$

$$A \rightarrow S$$

Eliminate  $S \rightarrow s$ :

$$K \rightarrow S$$

$$S \rightarrow ASB \mid aB \mid As \mid a \mid sB$$

$$A \rightarrow B \mid s$$

$$B \rightarrow b$$

Eliminate  $K \rightarrow S$ :

$$K \rightarrow ASB \mid aB \mid As \mid a \mid sB$$

$$S \rightarrow ASB \mid aB \mid As \mid a \mid sB$$

$$A \rightarrow B \mid s$$

$$B \rightarrow b$$

Eliminate  $A \rightarrow B$ :

$$K \rightarrow ASB \mid aB \mid As \mid a \mid sB$$

$$S \rightarrow ASB \mid aB \mid As \mid a \mid sB$$

$$A \rightarrow b \mid s$$

$$B \rightarrow b$$

Eliminate  $A \rightarrow s$ :

$$K \rightarrow ASB \mid aB \mid As \mid a \mid sB$$

$$S \rightarrow ASB \mid aB \mid As \mid a \mid sB$$

$$A \rightarrow b \mid ASB \mid aB \mid As \mid a \mid sB$$

$$B \rightarrow b$$

Step 3:

$$K \rightarrow ASB \mid xB \mid As \mid a \mid sB$$

$$S \rightarrow ASB \mid xB \mid As \mid a \mid sB$$

$$A \rightarrow b \mid ASB \mid xB \mid As \mid a \mid sB$$

$$B \rightarrow b$$

$$x \rightarrow a$$



Step 4:

$$k \rightarrow yB/xB/AS/a/SB$$

$$S \rightarrow yB/xB/AS/a/SB$$

$$A \rightarrow b/yB/xB/AS/a/SB$$

$$B \rightarrow b$$

$$x \rightarrow a$$

$$y \rightarrow AS \quad \text{(Converted CNF)}$$

Greibach Normal Form : (GNF)

- CFG can be said to be GNF, if it follows below rules:

Rules :

1. Initial production may have  $\epsilon$ -production

Ex:  $S \rightarrow \epsilon$

2. production on R.H.S may have only "one terminal"

ex:  $S \rightarrow a$

3. production on R.H.S may have ~~combination of~~ single terminal followed by any no. of non-terminals

ex:  $S \rightarrow aBA$

C.F.G. to G.N.F.:

- CFG can be converted to GNF by following below steps:

Steps:

1. Given grammar should be in CNF
2. Replace variable with numeric variable with ascending order

Ex:

$$A \rightarrow B$$

Sol

$$A \rightarrow C_1$$

$$B \rightarrow C_2$$

then

$$\boxed{C_1 \rightarrow C_2}$$

3. If  $A_i \rightarrow A_j$

$i < j$  (Leave for further substitution)

$i > j$  (Substitution)

$i = j$  (Left Recursion)

Example: Convert below given CFG to GNF

$$S \rightarrow AA|0$$

$$A \rightarrow ss|1$$

Sol: Step 1: It is already in CNF

Step 2:

$$\begin{array}{lcl} S \rightarrow A_1 & & A_1 \rightarrow A_2 A_2 | 0 \\ A \rightarrow A_2 & \Rightarrow & A_2 \rightarrow A_1 A_1 | 1 \end{array}$$

Step 3:  $A_i \rightarrow A_j$

consider  $A_1 \rightarrow A_2 A_2 | 0$  ~~is not~~

$$i=1, j=2$$

i.e.  $i < j$  (Leave for substitution)

consider  $A_2 \rightarrow A_1 A_1 | 1$

$$i=2, j=1$$

$i > j$  (substitution)

Substitute 'A<sub>1</sub>' in 'A<sub>2</sub>', then

$$A_2 \rightarrow A_1 A_1 | 1$$

$$A_2 \rightarrow A_2 A_2 A_1 | 1$$

$$i=2, j=2$$

$i=j$  (Left Recursion)

Left Recursion:

$$A_2 \rightarrow 0A_1Z \mid 1Z$$

$$Z \rightarrow A_2A_1Z \mid \epsilon$$

eliminate  $\epsilon$ :

$$\boxed{A_2 \rightarrow 0A_1Z \mid 1Z \mid 0A_1 \mid 1} \text{ in GNF}$$

$$Z \rightarrow A_2A_1Z \mid A_2A_1$$

Now Substitute ' $A_2$ ' in ' $A_1$ '

$$A_1 \rightarrow A_2A_2 \mid 0$$

$$\boxed{A_1 \rightarrow 0A_1Z A_2 \mid 1Z A_2 \mid 0A_1A_2 \mid 1A_2 \mid 0} \text{ in GNF}$$

Now substitute ' $A_2$ ' in ' $Z$ '

$$Z \rightarrow A_2A_1Z \mid A_2A_1$$

$$\boxed{\begin{aligned} Z \rightarrow & 0A_1Z A_1Z \mid 1Z A_1Z \mid 0A_1A_1Z \mid 1A_1Z \mid \\ & 0A_1Z A_1 \mid 1Z A_1 \mid 0A_1A_1 \mid 1A_1 \end{aligned}} \text{ in GNF}$$

Now Converted GNF grammar is:

$$A_1 \rightarrow 0A_1Z A_2 \mid 1Z A_2 \mid 0A_1A_2 \mid 1A_2 \mid 0$$

$$A_2 \rightarrow 0A_1Z \mid 1Z \mid 0A_1 \mid 1$$

$$\begin{aligned} Z \rightarrow & 0A_1Z A_1Z \mid 1Z A_1Z \mid 0A_1A_1Z \mid 1A_1Z \mid \\ & 0A_1Z A_1 \mid 1Z A_1 \mid 0A_1A_1 \mid 1A_1 \end{aligned}$$